

GPU Computing with CUDA

COMP 445

GPU: Massive fine-grain data parallelism

CPU

A relatively small amount of very fast, complex cores

Individual and shared cache and large main memory

Programming model is to split out many data elements per core

GP-GPU

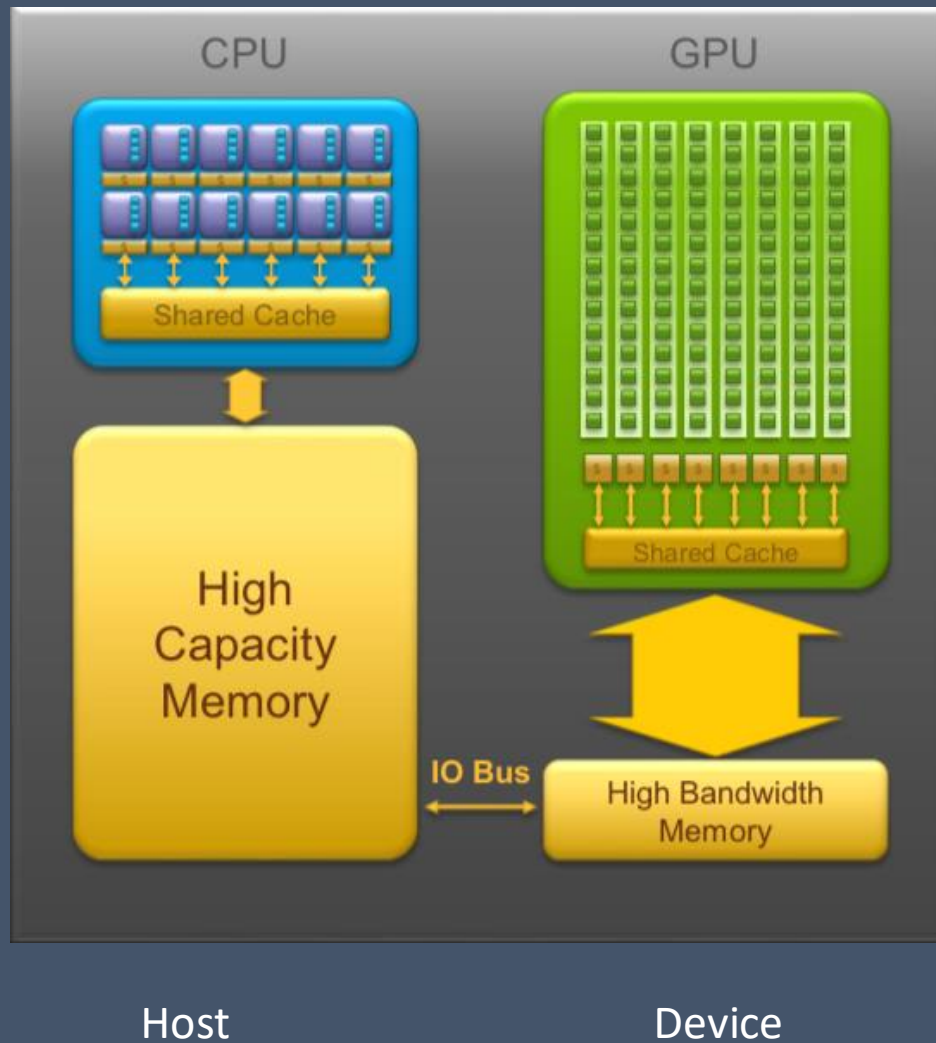
Thousands of simpler cores

Different memory layout

Different architecture

Programming model is one thread in one core works on one data element





Data must be transferred to/from the host memory

Kernel function runs on the device after data transfer

CPU memory is larger, GPU memory has more bandwidth

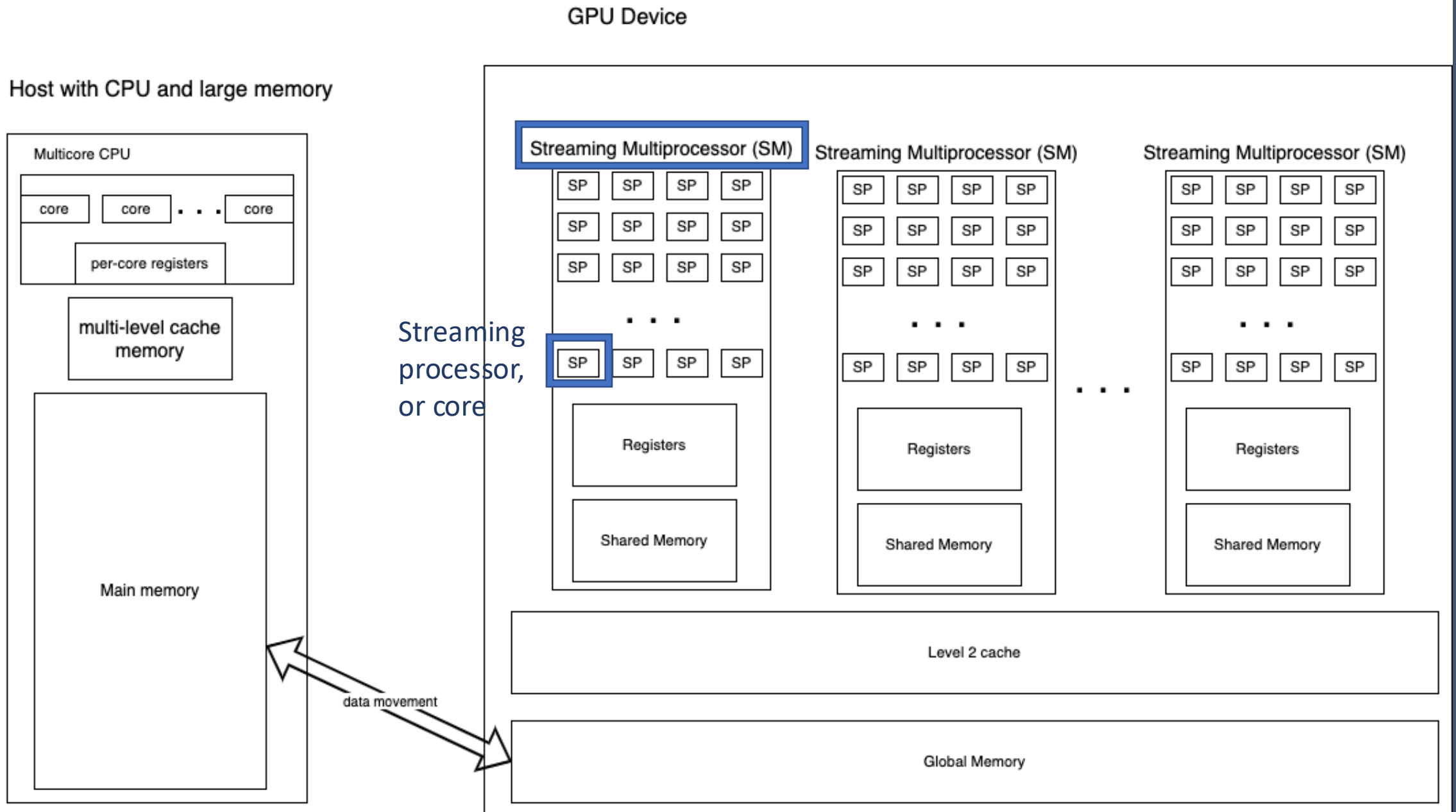
CPU and GPU memory are separate, connected by an I/O bus (traditionally PCI-e)

Any data transferred between the CPU and GPU will be handled by the I/O Bus

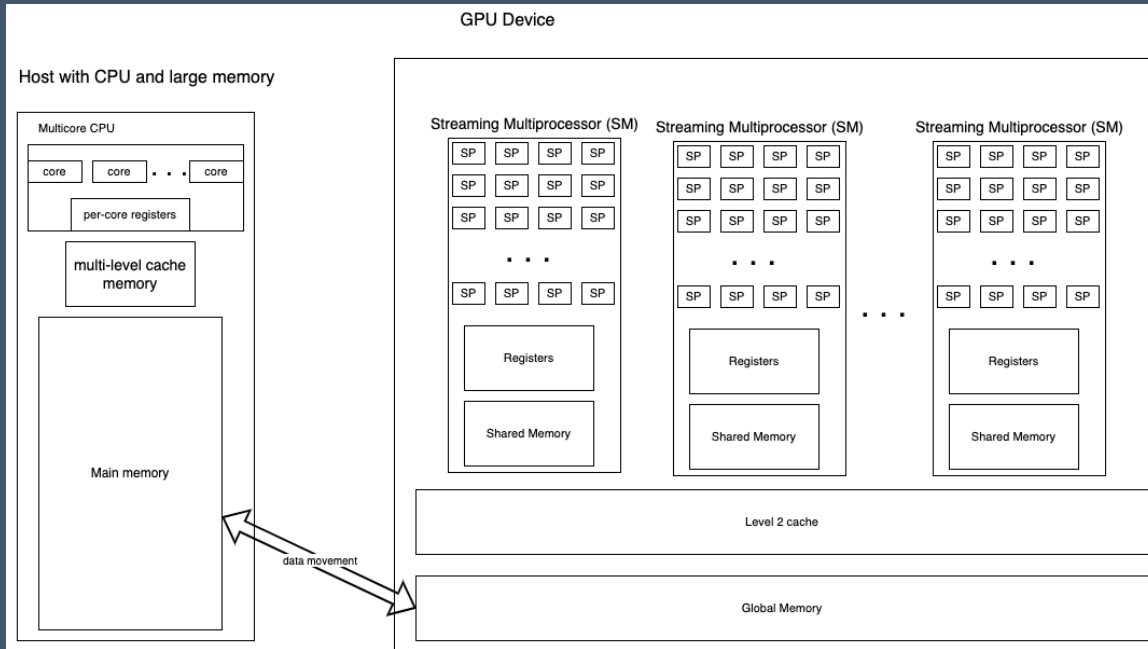
The I/O Bus is relatively slow compared to memory bandwidth

The GPU cannot perform computation until the data is within its memory

Different Architectures & Nomenclature



Different Architectures & Nomenclature



32 cores/SPs per SM

Each core running code gets registers

Some cores/SPs are for special calculations

SIMD (single instruction, multiple data):

When running instructions of a program, each SP in one SM is executing the same instruction

Warp scheduler: runs the code on an SM

CUDA Programming

Normally we will have large arrays of data

They can be managed in two ways:

1. Programmer manages (not used any more on newer cards):
 1. allocate memory on host and device,
 2. copy data to device,
 3. Compute on device,
 4. copy data back to host
2. Programmer specifies “unified memory”
 1. Allocate once as memory that will reside on device and host
 2. Compute on device

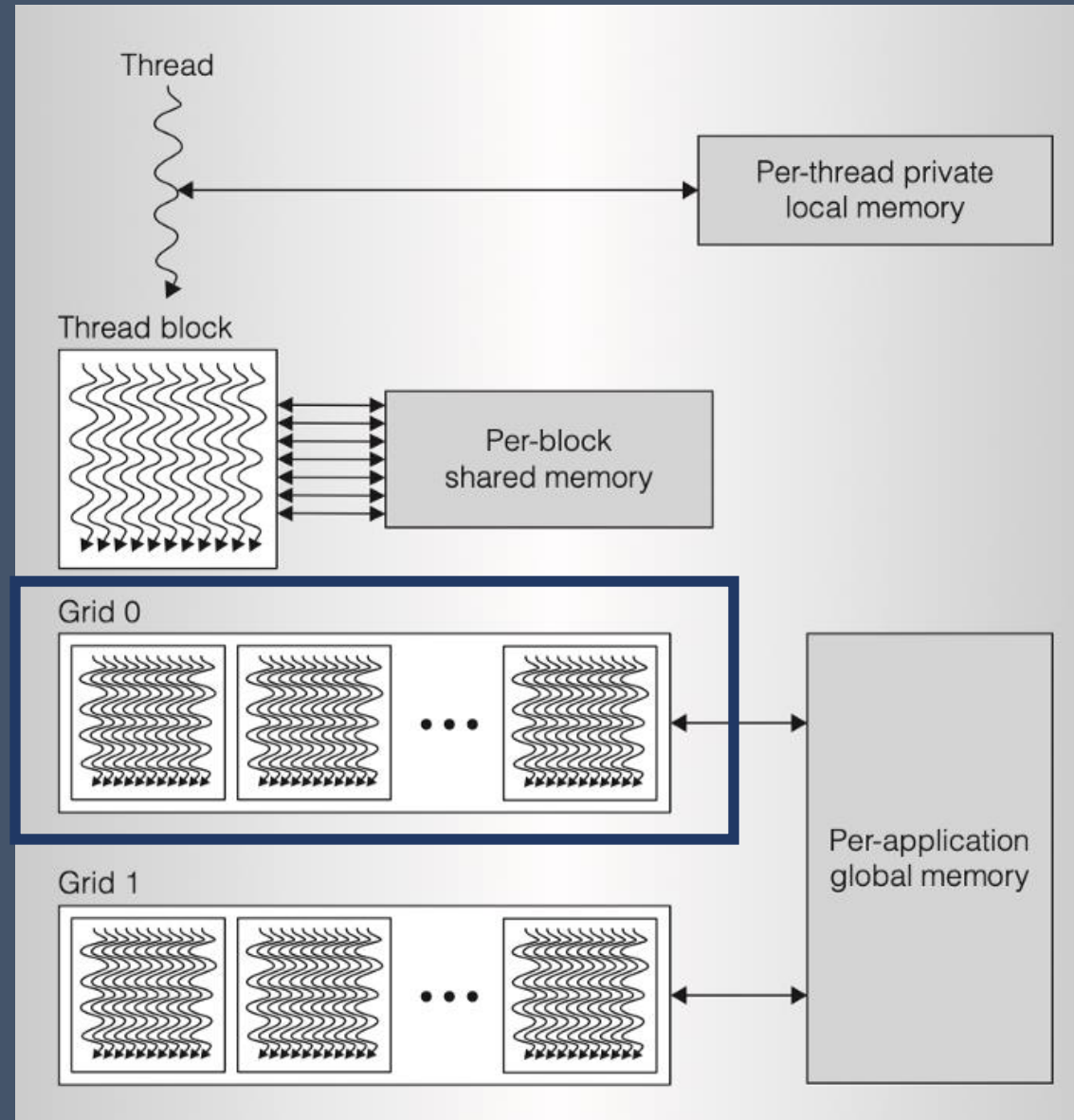
Programming Model

NVIDIA compiler:
CUDA

CUDA:
Programmer defines sizes:
Blocks per grid
Threads per block
Dimensions

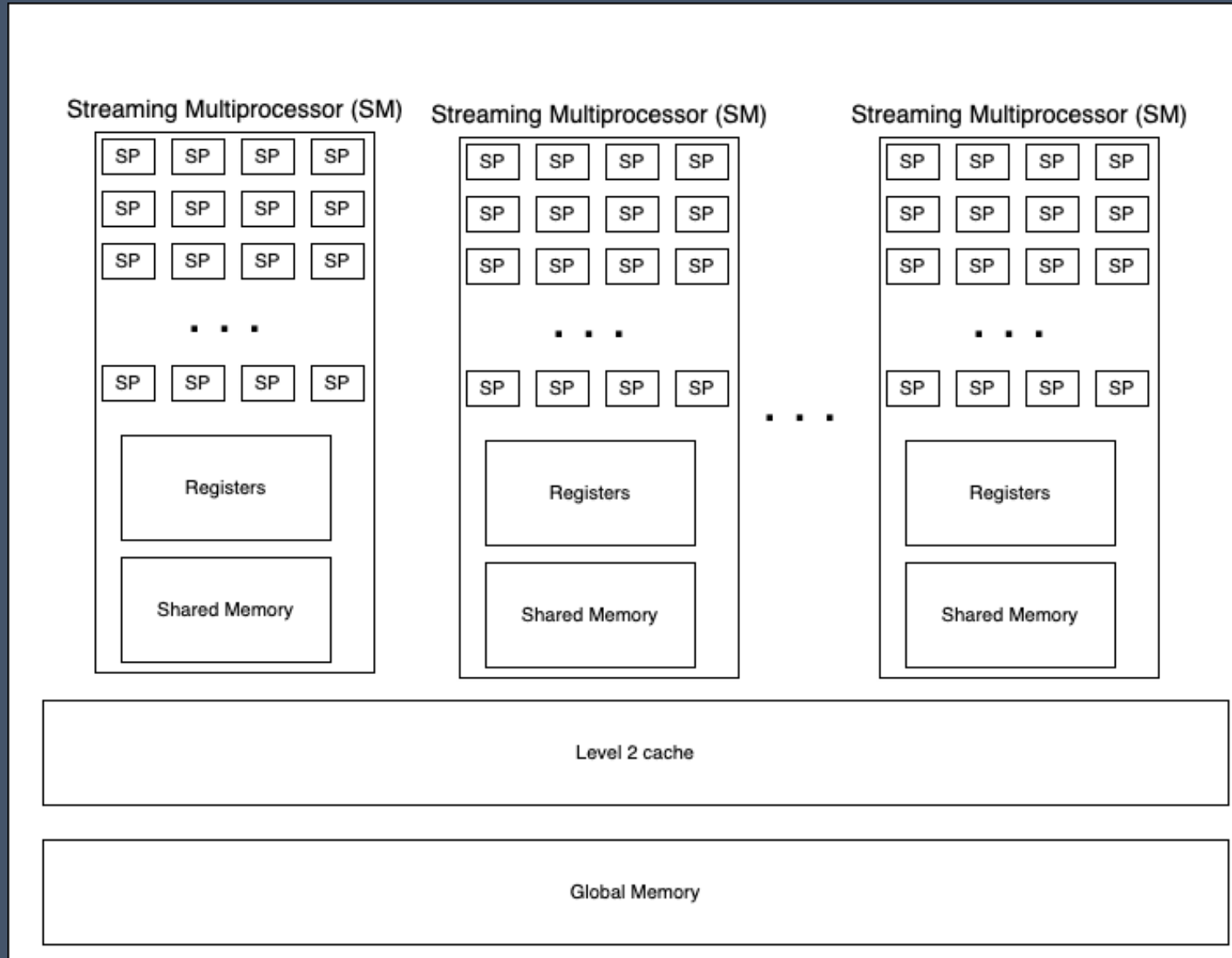
Simplest version:
1D blocks of threads
inside
1D grid of blocks

2D and 3D are possible



Taken from [this blog post](#)

NVIDIA Architecture



Programming
model of threads
is somewhat
related to this

CUDA Programming Model

1-D Grid of 1-D Blocks of threads: L Blocks in grid, N threads in a block

$\text{gridDim.x} = L$

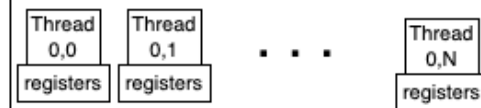
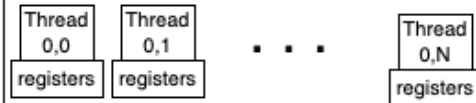
$\text{blockDim.x} = N$

$\text{blockIdx.y} = 0$

Block (0,0) $\text{blockIdx.x} = 0$

Block (0,L) $\text{blockIdx.x} = L$

$\text{threadIdx.y} = 0$



$\text{threadIdx.x} = 0$

$\text{threadIdx.x} = 1$

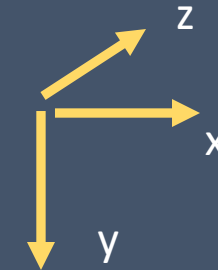
$\text{threadIdx.x} = N$

Shared Memory

Shared Memory

data movement

Global Memory



Programmer accessible per-thread values:

Overall dimensions:

gridDim.x (y, z)

blockDim.x (y, z)

Indexes of the block a thread is in:

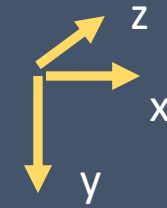
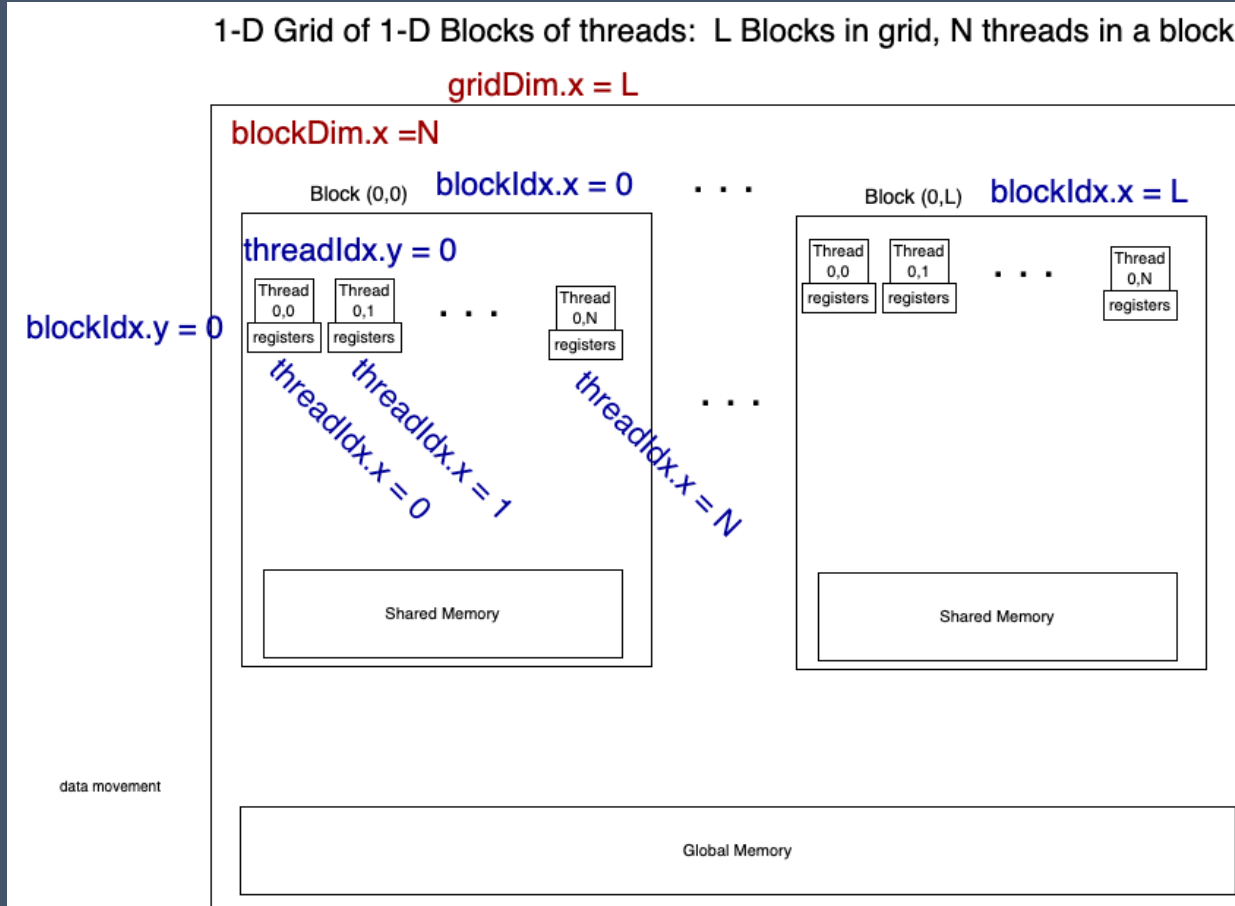
blockIdx.x (y, z)

Indexes of the position of the thread in a block:

threadIdx.x (y, z)

From these, a unique overall thread id can be calculated

CUDA Programming Model



Programmer accessible per-thread values:

Overall dimensions:

gridDim.x (y, z) ← Blocks per grid

blockDim.x (y, z) ← Threads per block

Indexes of the block a thread is in:

blockIdx.x (y, z)

Indexes of the position of the thread in a block:

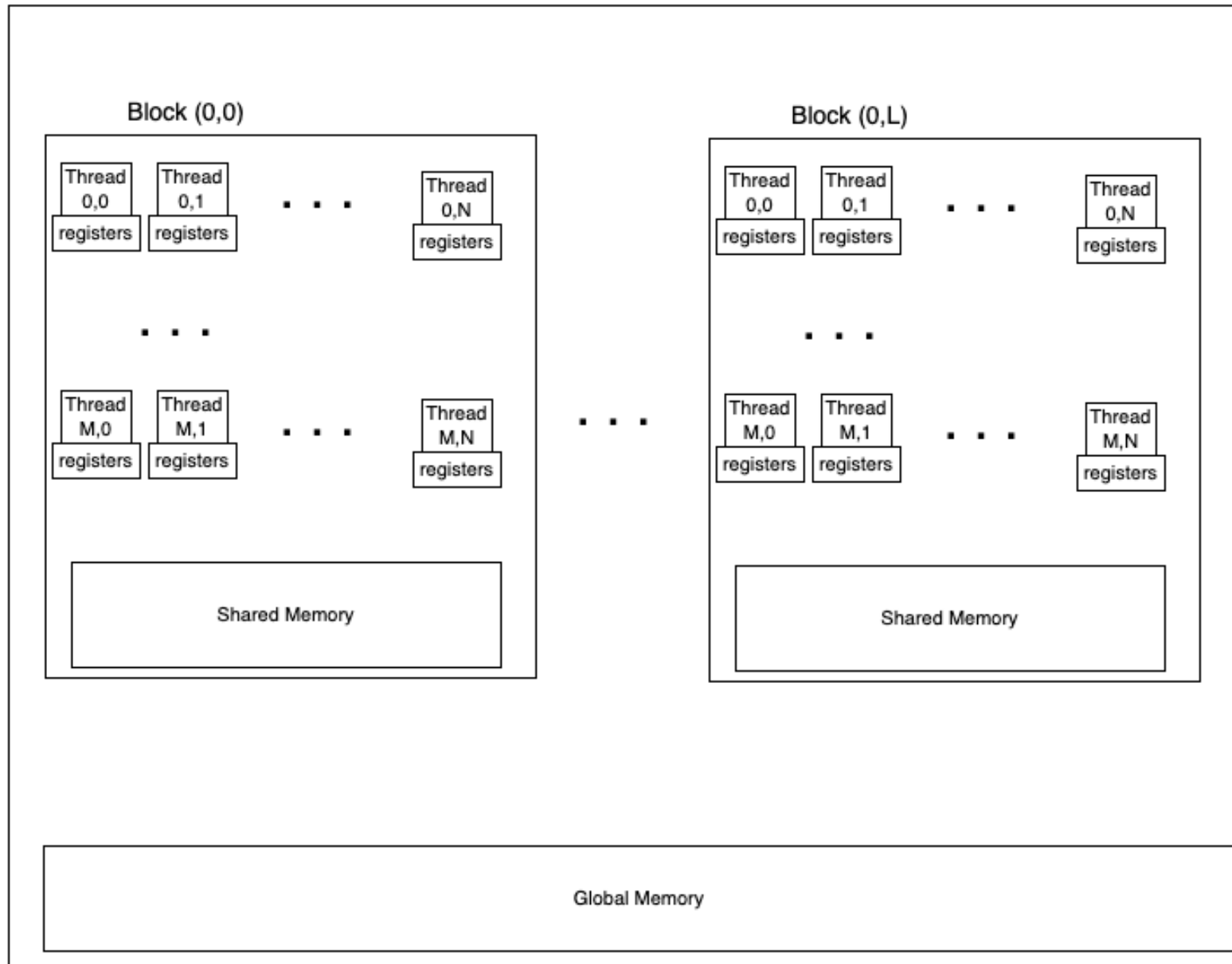
threadIdx.x (y, z)

Programmer determines unique thread id from 0 to (N x L)-1 like this:

((my block index in the x direction) * (number of threads in a block)) + my thread index in the block I'm in

$\text{threadNumber} = (\text{blockIdx.x} * \text{blockDim.x}) + \text{threadIdx.x}$

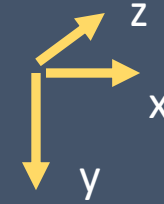
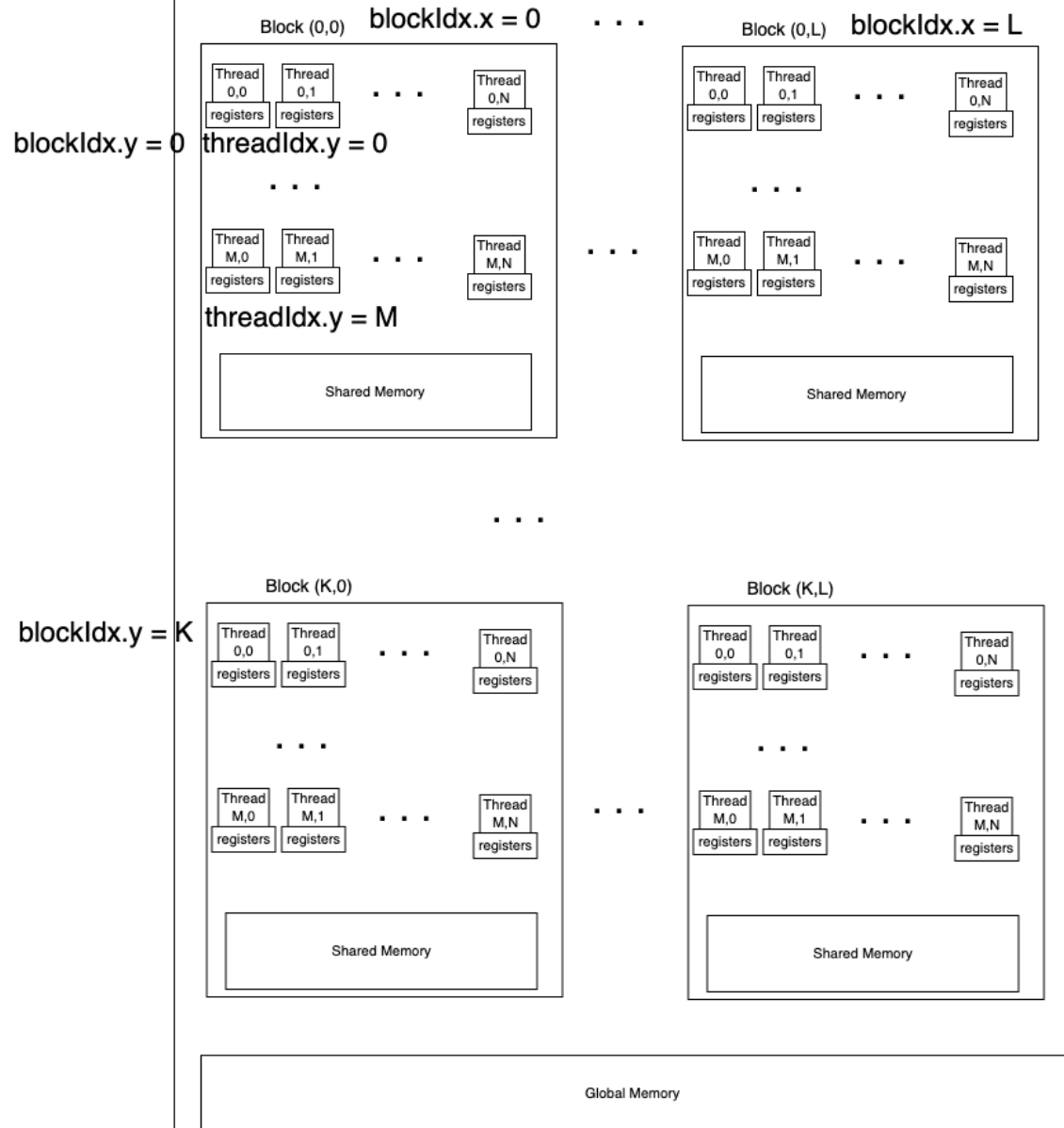
1-D Grid of 2-D Blocks of threads: L Blocks in grid, M x N threads in a block



2-D Grid of 2-D Blocks of threads: $K \times L$ Blocks in grid, $M \times N$ threads in a block

gridDim.x = L gridDim.y = K

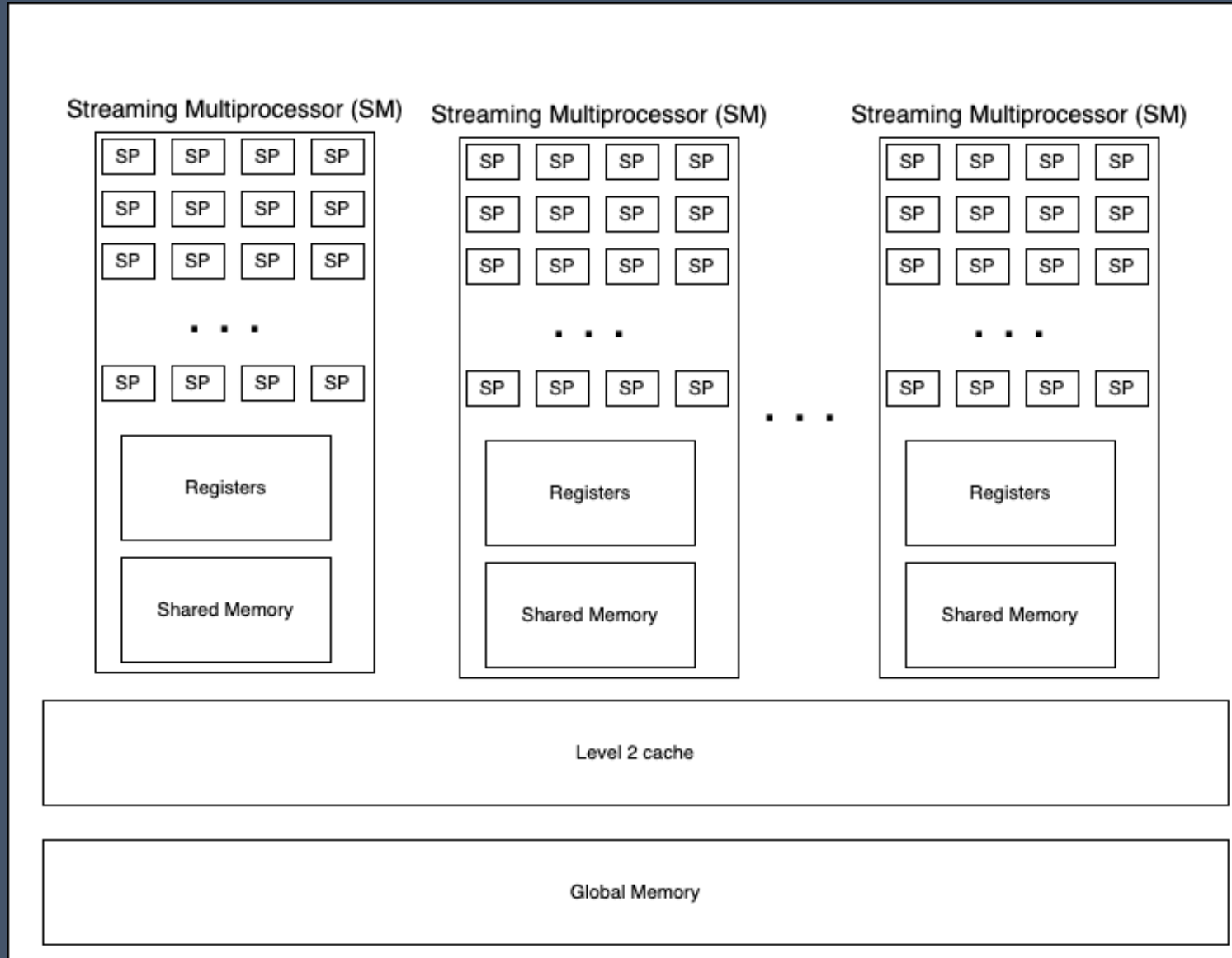
blockDim.x = N blockDim.y = M



NVIDIA Architecture

This
hasn't
changed:

32
SPs/cores
per SM



Programming
model of threads
is somewhat
related to this:

It's best to have
block sizes that
are a multiple of
32

Each block
specified maps to
an SM

CUDA Programming

Define a *kernel function* that will run on the device

```
__global__ void test1DGridKernel(int val) {  
    ...  
}
```

Call it by specifying how many blocks and how many threads per block (simplest case 1D grid of 1D blocks)

```
// 1 block of 32 threads goes to 1 SM  
test1DGridKernel<<<1, 32>>>(10);
```

CUDA Programming

Normally we will have large arrays of data

They must be managed in two ways:

1. Programmer manages:
 1. allocate memory on host and device,
 2. copy data to device,
 3. Compute on device,
 4. copy data back to host
2. Programmer specifies “unified memory”
 1. Allocate once as memory that will reside on device and host
 2. Compute on device